

Kontrol Alur Program: Mengatur Logika Percabangan dan Perulangan di Go

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Sebuah program yang statis hanya akan mengeksekusi kode dari baris teratas ke baris terbawah. Untuk membuat program yang cerdas, kita harus mampu mengendalikan arah jalannya eksekusi program berdasarkan kondisi tertentu (*Control Flow*) dan melakukan otomatisasi tugas berulang (*Looping*).

Dalam modul ini, kita akan membahas cara kerja pengambilan keputusan dengan `if`, `else`, dan `switch` di Go, serta mendalami satu-satunya keyword perulangan di Go, yaitu `for`, yang sangat fleksibel dan bertenaga.

Sesi 1: Logika Percabangan Tingkat Lanjut (If-Else) (45 Menit)

Percabangan `if` di Go memiliki sintaks yang mirip dengan bahasa pemrograman lain, namun dengan dua perbedaan utama: **tanpa tanda kurung (...) untuk kondisi** dan mendukung **deklarasi variabel temporer di dalam statement `if`**.

1. Sintaks Dasar If-Else

```
package main

import "fmt"

func main() {
    score := 85

    if score ≥ 90 {
        fmt.Println("Grade: A")
    } else if score ≥ 80 {
        fmt.Println("Grade: B")
    } else {
        fmt.Println("Grade: C")
    }
}
```

2. If dengan Temporary Variable (Go-Specific Pattern)

Salah satu fitur paling populer di Go adalah kemampuan mendeklarasikan variabel pembantu langsung di dalam baris statement `if` sebelum pengecekan kondisi dilakukan. Cakupan (*scope*) variabel tersebut hanya hidup di dalam blok `if` dan `else` terkait.

Pola ini sangat sering digunakan dalam penanganan error (*error handling*) dan operasi pencarian data:

```
package main

import (
    "fmt"
    "strconv"
)

func main() {
    // Format: if inisialisasi_variabel; kondisi_pengecekan { ... }
    if age, err := strconv.Atoi("25"); err != nil {
        fmt.Println("Gagal mengonversi umur:", err)
    } else {
        // Variabel 'age' hanya dapat diakses di dalam blok ini dan else-nya
        fmt.Printf("Konversi berhasil! Umur: %d tahun\n", age)
    }

    // Compile Error: fmt.Println(age) → age tidak dikenal di luar scope 'if'
}
```

TIP

Mengapa menggunakan temporary variable?

- **Meminimalkan Scope:** Variabel yang tidak diperlukan di bagian kode lain langsung dibuang dari memori saat blok `if` selesai.
- **Kode Lebih Bersih:** Menghindari tumpukan variabel tidak terpakai di scope fungsi utama.

Sesi 2: Logika Percabangan dengan Switch (35 Menit)

`switch` digunakan untuk menyederhanakan percabangan `if-else` bertingkat yang memeriksa satu nilai yang sama. Berbeda dengan C, C++, Java, atau PHP, Go memiliki sifat **auto-break**, artinya program tidak akan bocor ke case berikutnya secara tidak sengaja meskipun kita tidak menulis keyword `break`.

1. Switch Standar & Multiple Expressions

Setiap `case` dapat memiliki lebih dari satu nilai pencocokan yang dipisahkan oleh tanda koma:

```
package main

import "fmt"

func main() {
    day := "Sabtu"

    switch day {
    case "Senin", "Selasa", "Rabu", "Kamis", "Jumat":
        fmt.Println("Hari Kerja (Workday)")
    case "Sabtu", "Minggu":
        fmt.Println("Hari Libur (Weekend)")
    default:
        fmt.Println("Hari tidak valid")
    }
}
```

2. Switch Tanpa Ekspresi (Pengganti If-Else Rantai)

Jika kita tidak memberikan ekspresi setelah keyword `switch`, maka switch akan bertindak seperti `switch true`. Pola ini sangat bagus untuk menggantikan rangkaian `if-else if` yang kompleks agar lebih mudah dibaca:

```
package main

import "fmt"

func main() {
    backlogCount := 12

    switch {
    case backlogCount == 0:
        fmt.Println("Sistem aman, tidak ada antrean kerja.")
    case backlogCount > 0 && backlogCount ≤ 5:
        fmt.Println("Beban sistem: Ringan.")
    case backlogCount > 5 && backlogCount ≤ 10:
        fmt.Println("Beban sistem: Sedang. Perlu dipantau.")
    default:
        fmt.Println("⚠ BEBAN SISTEM TINGGI! Antrean menumpuk!")
    }
}
```

3. Kata Kunci `fallthrough`

Meskipun Go memiliki perilaku *auto-break*, kadang kala kita ingin memaksa eksekusi untuk berlanjut ke case berikutnya tanpa memedulikan kondisinya cocok atau tidak. Kita bisa menggunakan keyword `fallthrough` :

```
package main

import "fmt"

func main() {
    step := 2

    switch step {
    case 1:
        fmt.Println("Langkah 1: Inisialisasi Database")
        fallthrough
    case 2:
        fmt.Println("Langkah 2: Migrasi Skema Database")
        fallthrough
    case 3:
        fmt.Println("Langkah 3: Sinkronisasi Data Selesai")
    }
}

// Output jika step = 2:
// Langkah 2: Migrasi Skema Database
// Langkah 3: Sinkronisasi Data Selesai
```

Sesi 3: Perulangan dengan Kata Kunci Tunggal: `for` (45 Menit)

Salah satu filosofi kesederhanaan Go adalah meminimalkan jumlah keyword. Oleh karena itu, Go **tidak memiliki keyword** `while` atau `do-while` . Semua skenario perulangan di Go diselesaikan hanya dengan satu keyword: `for` .

1. Tiga Gaya Perulangan `for`

A. Gaya Standar (Three-Component Loop)

Menggunakan inisialisasi, kondisi evaluasi, dan post-statement.

```
for i := 1; i ≤ 5; i++ {
    fmt.Println("Iterasi ke-", i)
}
```

B. Gaya "While" (Single Condition Loop)

Inisialisasi diletakkan di luar, dan increment diletakkan di dalam blok kode. Hanya kondisi pengecekan yang ditaruh setelah keyword `for`.

```
counter := 1
for counter ≤ 5 {
    fmt.Println("Nilai counter:", counter)
    counter++
}
```

C. Gaya "Infinite Loop" (Tanpa Kondisi)

Sangat berguna untuk proses background worker yang harus berjalan terus menerus (misalnya antrean pesan MQ atau tailing log file). Kita menggunakan keyword `break` untuk keluar dari loop.

```
package main

import "fmt"

func main() {
    counter := 1
    for {
        fmt.Println("Loop berjalan...", counter)
        if counter ≥ 5 {
            break // Keluar paksa dari perulangan
        }
        counter++
    }
}
```

2. Iterasi Kumpulan Data dengan `for range`

Untuk membaca atau memproses data di dalam array, slice, map, atau karakter string, Go menyediakan konstruksi `for range` yang sangat bersih:

```

package main

import "fmt"

func main() {
    // Slice data string
    logLevels := []string{"INFO", "WARN", "ERROR", "FATAL"}

    // index dan value dikembalikan secara bersamaan
    for idx, val := range logLevels {
        fmt.Printf("Indeks: %d, Level Log: %s\n", idx, val)
    }

    // Gunakan blank identifier (_) jika tidak membutuhkan indeks
    for _, val := range logLevels {
        fmt.Println("Tingkat Bahaya:", val)
    }
}

```

3. Pengendalian Loop: `break`, `continue`, dan `Label`

- `break` : Berfungsi untuk menghentikan dan keluar dari loop secara instan.
- `continue` : Menghentikan iterasi saat ini dan langsung melompat ke iterasi berikutnya.

Loop Bersarang & Label (Nested Loop Break)

Di level menengah, Anda akan sering berhadapan dengan *nested loop* (loop di dalam loop). Jika Anda memanggil `break` di dalam loop terdalam, Go hanya akan menghentikan loop terdalam tersebut.

Untuk menghentikan *seluruh* tingkat perulangan (termasuk parent loop), kita harus mendefinisikan sebuah **Label**:

```

package main

import "fmt"

func main() {
OuterLoop: // Mendefinisikan label luar
    for i := 1; i ≤ 3; i++ {
        for j := 1; j ≤ 3; j++ {
            if i = 2 && j = 2 {
                fmt.Println("Kombinasi kritis ditemukan! Menghentikan se
                break OuterLoop // Menghentikan OuterLoop, bukan hanya l
            }
            fmt.Printf("Menyinkronkan Node %d - Partisi %d\n", i, j)
        }
    }
    fmt.Println("Selesai.")
}

```

--## Sesi 4: Praktikum Mandiri: "Sistem Checkout Keranjang & Verifikasi Transaksi" (40 Menit)

Mari kita satukan logika percabangan, perulangan, `switch`, `for-range`, dan penanganan label dalam sebuah simulasi praktikum dunia nyata. Kita akan membuat **Sistem Validasi & Checkout Keranjang Belanja E-commerce**.

Skenario Kasus:

1. Program membaca daftar item di keranjang belanja dengan format data string gabungan.
2. Program memproses setiap item menggunakan `for range` dan memecah data untuk mendapatkan nama produk, harga, kuantitas, dan status ketersediaan stok.
3. Program memeriksa stok menggunakan `switch-case`. Jika status stok adalah `"out_of_stock"`, program akan melewati (*skip*) item tersebut menggunakan keyword `continue`.
4. Jika total belanja untuk satu item melebihi Rp 10.000.000 (Transaksi Risiko Tinggi), program akan memicu prosedur verifikasi OTP tambahan sebanyak maksimal 3 kali percobaan (*limit retry*) menggunakan perulangan `for`.
5. Jika verifikasi OTP gagal sebanyak 3 kali, transaksi dianggap mencurigakan (*fraud*). Seluruh proses checkout keranjang belanja akan dibatalkan seketika menggunakan **Label Break**.

Implementasi Kode (`main.go`)

Buatlah proyek baru atau edit berkas `main.go` Anda, lalu tulis kode di bawah ini:

```

package main

import (
    "fmt"
    "strings"
    "time"
)

func main() {
    // 1. Data dummy item belanja (nama_produk | harga_satuan | jumlah_beli | status)
    cartItems := []string{
        "Laptop Asus ROG | 25000000 | 1 | ready",
        "Mouse Wireless | 350000 | 2 | ready",
        "Smartphone Samsung | 12000000 | 1 | out_of_stock",
        "T-Shirt Basic | 150000 | 3 | ready",
    }

    fmt.Println("=====")
    fmt.Println("          MEMULAI PROSES CHECKOUT KERANJANG          ")
    fmt.Println("=====")

    var totalBelanja float64

    // Label untuk menghentikan seluruh checkout jika terjadi indikasi fraud
    CheckoutProcessing:
    for idx, item := range cartItems {
        // Pecah data string berdasarkan pembatas " | "
        parts := strings.Split(item, " | ")
        productName := parts[0]

        // Parsing harga dan jumlah dari string
        var price float64
        var qty int
        fmt.Sscanf(parts[1], "%f", &price)
        fmt.Sscanf(parts[2], "%d", &qty)
        stockStatus := parts[3]

        fmt.Printf("\n[Item #%d] Memproses: %s x%d\n", idx+1, productName, qty)

        // Percabangan Switch-Case untuk memeriksa stok
        switch stockStatus {
            case "out_of_stock":
                fmt.Println(">> Status: STOK HABIS! Melewati item ini.")
                continue // Lompat ke perulangan item berikutnya (for range)

            case "ready":
                subtotal := price * float64(qty)
                totalBelanja += subtotal
                fmt.Printf(">> Status: Ready. Subtotal: Rp %.2f\n", subtotal)

                // Logika Bisnis: Proteksi Transaksi Nilai Tinggi (> Rp 10.000.000)
                if subtotal > 10000000.0 {

```



```

        fmt.Println(">> Peringatan: Transaksi Risiko Tinggi Terdeteksi")
        fmt.Println(">> Memulai prosedur verifikasi OTP Keamanan")

        maxRetries := 3
        isOtpSuccess := false

        // Perulangan retry OTP
        for attempt := 1; attempt ≤ maxRetries; attempt++ {
            fmt.Printf("    → Percobaan OTP %d/%d: Mengirimkan OTP ke HP\n")

            // Simulasi: Percobaan 1 & 2 gagal (user salah memasukkan OTP)
            if attempt = 3 {
                isOtpSuccess = true
            }

            // Jeda simulasi proses
            time.Sleep(500 * time.Millisecond)

            if isOtpSuccess {
                fmt.Println("    [OK] Verifikasi OTP Sukses")
                break // Keluar dari loop retry OTP
            } else {
                fmt.Println("    [FAILED] Kode OTP tidak valid")
            }
        }

        // Jika verifikasi OTP gagal total setelah 3 kali mencoba
        if !isOtpSuccess {
            fmt.Println("\n>> FRAUD DETECTED: Verifikasi OTP Gagal")
            fmt.Println(">> Membatalkan seluruh transaksi checkout")
            totalBelanja = 0 // Reset nilai belanja
            break CheckoutProcessing // Batalkan seluruh checkout
        }
    }

    }

    fmt.Println("\n=====")
    if totalBelanja > 0 {
        fmt.Printf("    TOTAL TRANSAKSI : Rp %15.2f\n", totalBelanja)
        fmt.Println("    STATUS CHECKOUT : BERHASIL!")
    } else {
        fmt.Println("    STATUS CHECKOUT : DIBATALKAN / TIDAK ADA ITEM VALID")
    }
    fmt.Println("=====")
}

```

Sesi 5: Tantangan Mandiri (45 Menit)

Guna menguji kematangan logika Anda terkait materi `if-else`, `switch`, dan perulangan `for`, selesaikan tantangan mandiri berikut:

Tantangan: Sistem Loyalty Points & Voucher Cashback

Modifikasi kode praktikum e-commerce di atas agar memiliki kalkulator rewards untuk pelanggan setia setelah checkout berhasil.

Kriteria Tantangan:

1. Definisikan aturan perolehan poin: Pelanggan berhak mendapatkan **10 poin untuk setiap kelipatan Rp 100.000** dari total transaksi final (Gunakan operator aritmatika `/` dan `*` dengan konversi tipe data yang sesuai).
2. Gunakan percabangan **Switch Tanpa Ekspresi** (atau `if-else`) untuk mendeteksi besaran bonus cashback voucher belanja berdasarkan total belanja akhir:
 - Total Belanja > Rp 20.000.000: Mendapatkan Voucher Cashback sebesar Rp 1.000.000.
 - Total Belanja > Rp 5.000.000: Mendapatkan Voucher Cashback sebesar Rp 200.000.
 - Total Belanja Lainnya: Tidak mendapatkan voucher cashback.
3. Tampilkan hasil ringkasan perolehan poin dan bonus voucher cashback tersebut di akhir program menggunakan format tabel struk yang rapi dengan `fmt.Printf`.
4. Ujilah program Anda dengan mengganti atau mengacak data input di dalam array `cartItems` untuk melihat apakah sistem kalkulasi reward Anda merespons perubahan total belanja secara dinamis dan akurat.

Sesi 6: Latihan Soal Praktik Tambahan (E-Commerce Context)

Gunakan seluruh pemahaman Anda mengenai `if-else`, `switch`, `for`, `range`, dan `break/continue` untuk menyelesaikan dua skenario kompleks di bawah ini.

Latihan 1: Engine Kalkulator Diskon Bertingkat & Membership Premium

Skenario: Sebuah platform e-commerce ingin mengotomatiskan pemberian diskon berdasarkan kategori produk, status membership, dan total belanja.

Spesifikasi Program:

1. **Data Input:** Buat slice of string `items` dengan format `"Nama | Kategori | Harga"`.

- Kategori: `Electronics` , `Fashion` , `Grocery` .

2. Aturan Diskon Kategori:

- `Electronics` : Diskon 10% jika harga > 5.000.000.
- `Fashion` : Diskon 20% jika harga > 500.000.
- `Grocery` : Diskon 5% jika membeli lebih dari 5 item (simulasikan dengan kuantitas).

3. Bonus Membership:

- Jika user adalah `PREMIUM` , berikan tambahan diskon 5% dari total belanja akhir (setelah diskon kategori).
- Jika user adalah `GOLD` , berikan tambahan diskon 10% dari total belanja akhir.

4. Threshold Gratis Ongkir: Jika total belanja akhir > 2.000.000, tampilkan pesan "Selamat! Anda mendapatkan Gratis Ongkir".

Tugas: Buatlah program yang memproses daftar belanja tersebut dan mencetak struk rincian harga awal, potongan diskon kategori, potongan membership, dan total akhir yang harus dibayar.

Latihan 2: Simulasi Sistem Flash Sale & Limit Stock (Advanced Loop)

Skenario: Anda diminta membangun logika sistem Flash Sale sederhana. Terdapat stok terbatas yang diperebutkan oleh banyak calon pembeli dalam waktu yang sangat singkat.

Spesifikasi Program:

1. **Inisialisasi:** Stok awal produk adalah 10 unit.
2. **Data Pembeli:** Buat slice of string berisi nama-nama calon pembeli (minimal 15 orang).
3. **Proses Pembelian:**
 - Gunakan perulangan `for-range` untuk memproses setiap pembeli.
 - Setiap pembeli memiliki "Waktu Respon" (simulasikan dengan angka random 1-100ms menggunakan `math/rand`).
 - Jika Waktu Respon pembeli > 80ms, anggap koneksi mereka lambat dan tampilkan pesan "Gagal: Koneksi Timeout". Gunakan `continue` .
 - Jika stok masih tersedia, kurangi stok dan tampilkan "Berhasil: [Nama] mendapatkan produk!".
 - Jika stok habis (0), tampilkan "Sold Out: [Nama] kehabisan stok!".
 - Gunakan `break` untuk segera menghentikan seluruh proses perulangan jika stok sudah benar-benar 0 agar tidak memproses sisa pembeli yang sudah pasti tidak akan dapat.

Tugas: Implementasikan logika tersebut dan tampilkan ringkasan di akhir: Berapa produk yang terjual dan siapa saja pembeli yang beruntung.